

Deep Detect

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 Namespace Index	3
2.1 Package List	3
3 File Index	5
3.1 File List	5
4 Directory Documentation	7
4.1 src Directory Reference	7
4.2 utils Directory Reference	7
5 Namespace Documentation	9
5.1 ai_detection Namespace Reference	9
5.2 app Namespace Reference	9
5.2.1 Function Documentation	10
5.2.1.1 load_ai_models()	10
5.2.2 Variable Documentation	10
5.2.2.1 _	10
5.2.2.2 ai_image	10
5.2.2.3 akaze	11
5.2.2.4 area	11
5.2.2.5 c1	11
5.2.2.6 c2	11
5.2.2.7 c3	11
5.2.2.8 caption	11
5.2.2.9 cnn_fake_score	11
5.2.2.10 cnn_input	11
5.2.2.11 cnn_model	11
5.2.2.12 cnn_prediction	11
5.2.2.13 cnn_real_score	12
5.2.2.14 col1	12
5.2.2.15 col2	12
5.2.2.16 col3	12
5.2.2.17 contours	12
5.2.2.18 des_akaze	12
5.2.2.19 des_orb	12
5.2.2.20 des_sift	12
5.2.2.21 des_surf	12
5.2.2.22 edges	12
5.2.2.23 final_score	13
5.2.2.24 gray	13
5.2.2.25 h	13

5.2.2.26 image	13
5.2.2.27 img	13
5.2.2.28 img_akaze	13
5.2.2.29 img_orb	13
5.2.2.30 img_sift	13
5.2.2.31 IMG_SIZE	13
5.2.2.32 img_surf	13
5.2.2.33 kp_akaze	14
5.2.2.34 kp_orb	14
5.2.2.35 kp_sift	14
5.2.2.36 kp_surf	14
5.2.2.37 layout	14
5.2.2.38 lstm_fake_score	14
5.2.2.39 lstm_input	14
5.2.2.40 lstm_model	14
5.2.2.41 lstm_prediction	14
5.2.2.42 lstm_real_score	14
5.2.2.43 marked_img	15
5.2.2.44 orb	15
5.2.2.45 page_icon	15
5.2.2.46 page_title	15
5.2.2.47 sift	15
5.2.2.48 surf	15
5.2.2.49 suspicious_count	15
5.2.2.50 tab1	15
5.2.2.51 tab2	15
5.2.2.52 tab3	15
5.2.2.53 tab4	16
5.2.2.54 unsafe_allow_html	16
5.2.2.55 uploaded_file	16
5.2.2.56 w	16
5.2.2.57 width	16
5.2.2.58 x	16
5.2.2.59 y	16
5.3 evaluate_models Namespace Reference	16
5.3.1 Function Documentation	17
5.3.1.1 evaluate_folder()	17
5.3.1.2 predict_image()	17
5.3.1.3 prepare_image()	17
5.3.2 Variable Documentation	17
5.3.2.1 cnn_model	17
5.3.2.2 IMG_SIZE	17

5.3.2.3 limit	17
5.3.2.4 lstm_model	17
5.4 feature_detection Namespace Reference	18
5.5 image_loader Namespace Reference	18
5.6 manual_model_check Namespace Reference	18
5.6.1 Function Documentation	18
5.6.1.1 prepare_image()	18
5.6.1.2 test_folder()	18
5.6.2 Variable Documentation	18
5.6.2.1 cnn_model	18
5.6.2.2 fake_index	18
5.6.2.3 IMG_SIZE	19
5.6.2.4 lstm_model	19
5.7 metrics Namespace Reference	19
5.7.1 Function Documentation	19
5.7.1.1 calculate_fake_ratio()	19
5.7.1.2 interpret_result()	19
5.8 model_test Namespace Reference	19
5.8.1 Function Documentation	19
5.8.1.1 test_app_file_exists()	19
5.9 test_utils Namespace Reference	20
5.9.1 Function Documentation	20
5.9.1.1 test_calculate_fake_ratio()	20
5.9.1.2 test_interpret_result()	20
5.10 train_lstm Namespace Reference	20
5.10.1 Variable Documentation	21
5.10.1.1 base_model	21
5.10.1.2 BATCH_SIZE	21
5.10.1.3 class_weight	21
5.10.1.4 class_weights	21
5.10.1.5 class_weights_array	21
5.10.1.6 classes	21
5.10.1.7 DATASET_DIR	21
5.10.1.8 EPOCHS	21
5.10.1.9 epochs	22
5.10.1.10 fake_index	22
5.10.1.11 IMG_SIZE	22
5.10.1.12 loss	22
5.10.1.13 metrics	22
5.10.1.14 model	22
5.10.1.15 MODEL_DIR	22
5.10.1.16 optimizer	22

5.10.1.17 output	22
5.10.1.18 train_datagen	23
5.10.1.19 train_generator	23
5.10.1.20 trainable	23
5.10.1.21 val_generator	23
5.10.1.22 validation_data	23
5.10.1.23 x	23
5.11 train_models Namespace Reference	24
5.11.1 Variable Documentation	24
5.11.1.1 base_model	24
5.11.1.2 BATCH_SIZE	24
5.11.1.3 class_weight	24
5.11.1.4 class_weights	25
5.11.1.5 class_weights_array	25
5.11.1.6 classes	25
5.11.1.7 DATASET_DIR	25
5.11.1.8 epochs	25
5.11.1.9 EPOCHS_FINE	25
5.11.1.10 EPOCHS_HEAD	25
5.11.1.11 exist_ok	25
5.11.1.12 fake_index	25
5.11.1.13 IMG_SIZE	26
5.11.1.14 loss	26
5.11.1.15 metrics	26
5.11.1.16 model	26
5.11.1.17 MODEL_DIR	26
5.11.1.18 optimizer	26
5.11.1.19 output	26
5.11.1.20 train_datagen	26
5.11.1.21 train_generator	27
5.11.1.22 trainable	27
5.11.1.23 val_generator	27
5.11.1.24 validation_data	27
5.11.1.25 x	27
5.12 utils Namespace Reference	27
6 File Documentation	29
6.1 app.py File Reference	29
6.2 evaluate_models.py File Reference	30
6.3 manual_model_check.py File Reference	31
6.4 model_test.py File Reference	31
6.5 README.md File Reference	31

6.6 src/ai_detection.py File Reference	31
6.7 src/feature_detection.py File Reference	32
6.8 src/image_loader.py File Reference	32
6.9 src/utils.py File Reference	32
6.10 test_utils.py File Reference	32
6.11 train_lstm.py File Reference	32
6.12 train_models.py File Reference	33
6.13 utils/metrics.py File Reference	34
Index	35

Chapter 1

Directory Hierarchy

1.1 Directories

src	7
ai_detection.py	31
feature_detection.py	32
image_loader.py	32
utils.py	32
utils	7
metrics.py	34

Chapter 2

Namespace Index

2.1 Package List

Here are the packages with brief descriptions (if available):

ai_detection	9
app	9
evaluate_models	16
feature_detection	18
image_loader	18
manual_model_check	18
metrics	19
model_test	19
test_utils	20
train_lstm	20
train_models	24
utils	27

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

app.py	29
evaluate_models.py	30
manual_model_check.py	31
model_test.py	31
test_utils.py	32
train_lstm.py	32
train_models.py	33
src/ai_detection.py	31
src/feature_detection.py	32
src/image_loader.py	32
src/utils.py	32
utils/metrics.py	34

Chapter 4

Directory Documentation

4.1 src Directory Reference

Files

- file [ai_detection.py](#)
- file [feature_detection.py](#)
- file [image_loader.py](#)
- file [utils.py](#)

4.2 utils Directory Reference

Files

- file [metrics.py](#)

Chapter 5

Namespace Documentation

5.1 ai_detection Namespace Reference

5.2 app Namespace Reference

Functions

- [load_ai_models\(\)](#)

Variables

- [page_title](#)
- [page_icon](#)
- [layout](#)
- [int IMG_SIZE = 224](#)
- [cnn_model](#)
- [lstm_model](#)
- [unsafe_allow_html](#)
- [uploaded_file](#)
- [image](#) = `Image.open(uploaded_file).convert("RGB")`
- [img](#) = `np.array(image)`
- [gray](#) = `cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)`
- [c1](#)
- [c2](#)
- [c3](#)
- [caption](#)
- [width](#)
- [tab1](#)
- [tab2](#)
- [tab3](#)
- [tab4](#)
- [orb](#) = `cv2.ORB_create()`
- [kp_orb](#)
- [des_orb](#)
- [img_orb](#) = `cv2.drawKeypoints(img, kp_orb, None, color=(0, 255, 0))`
- [akaze](#) = `cv2.AKAZE_create()`

- [kp_akaze](#)
- [des_akaze](#)
- [img_akaze](#) = cv2.drawKeypoints([img](#), [kp_akaze](#), None, color=(255, 0, 0))
- [sift](#) = cv2.SIFT_create()
- [kp_sift](#)
- [des_sift](#)
- [img_sift](#) = cv2.drawKeypoints([img](#), [kp_sift](#), None, color=(0, 0, 255))
- [surf](#) = cv2.xfeatures2d.SURF_create(400)
- [kp_surf](#)
- [des_surf](#)
- [img_surf](#) = cv2.drawKeypoints([img](#), [kp_surf](#), None, color=(255, 255, 0))
- [edges](#) = cv2.Canny([gray](#), 100, 200)
- [contours](#)
- [_](#)
- [marked_img](#) = [img](#).copy()
- [int_suspicious_count](#) = 0
- [area](#) = cv2.contourArea([contour](#))
- [x](#)
- [y](#)
- [w](#)
- [h](#)
- [ai_image](#) = cv2.resize([img](#), ([IMG_SIZE](#), [IMG_SIZE](#)))
- [cnn_input](#) = np.expand_dims([ai_image](#), axis=0)
- [lstm_input](#) = np.expand_dims([ai_image](#), axis=0)
- [cnn_prediction](#) = [cnn_model](#).predict([cnn_input](#), verbose=0)
- [lstm_prediction](#) = [lstm_model](#).predict([lstm_input](#), verbose=0)
- [int_cnn_real_score](#) = [cnn_prediction](#)[0][0] * 100
- [int_cnn_fake_score](#) = [cnn_prediction](#)[0][1] * 100
- [int_lstm_real_score](#) = [lstm_prediction](#)[0][0] * 100
- [int_lstm_fake_score](#) = [lstm_prediction](#)[0][1] * 100
- [tuple_final_score](#) = ([cnn_fake_score](#) * 0.7) + ([lstm_fake_score](#) * 0.3)
- [col1](#)
- [col2](#)
- [col3](#)

5.2.1 Function Documentation

5.2.1.1 load_ai_models()

```
app.load_ai_models ()
```

5.2.2 Variable Documentation

5.2.2.1 _

```
app._ [protected]
```

5.2.2.2 ai_image

```
app.ai_image = cv2.resize(img, (IMG\_SIZE, IMG\_SIZE))
```

5.2.2.3 akaze

```
app.akaze = cv2.AKAZE_create()
```

5.2.2.4 area

```
app.area = cv2.contourArea(contour)
```

5.2.2.5 c1

```
app.c1
```

5.2.2.6 c2

```
app.c2
```

5.2.2.7 c3

```
app.c3
```

5.2.2.8 caption

```
app.caption
```

5.2.2.9 cnn_fake_score

```
int app.cnn_fake_score = cnn_prediction[0][1] * 100
```

5.2.2.10 cnn_input

```
app.cnn_input = np.expand_dims(ai_image, axis=0)
```

5.2.2.11 cnn_model

```
app.cnn_model
```

5.2.2.12 cnn_prediction

```
app.cnn_prediction = cnn_model.predict(cnn_input, verbose=0)
```

5.2.2.13 `cnn_real_score`

```
int app.cnn_real_score = cnn_prediction[0][0] * 100
```

5.2.2.14 `col1`

```
app.col1
```

5.2.2.15 `col2`

```
app.col2
```

5.2.2.16 `col3`

```
app.col3
```

5.2.2.17 `contours`

```
app.contours
```

5.2.2.18 `des_akaze`

```
app.des_akaze
```

5.2.2.19 `des_orb`

```
app.des_orb
```

5.2.2.20 `des_sift`

```
app.des_sift
```

5.2.2.21 `des_surf`

```
app.des_surf
```

5.2.2.22 `edges`

```
app.edges = cv2.Canny(gray, 100, 200)
```

5.2.2.23 final_score

```
tuple app.final_score = (cnn_fake_score * 0.7) + (lstm_fake_score * 0.3)
```

5.2.2.24 gray

```
app.gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
```

5.2.2.25 h

```
app.h
```

5.2.2.26 image

```
app.image = Image.open(uploaded_file).convert("RGB")
```

5.2.2.27 img

```
app.img = np.array(image)
```

5.2.2.28 img_akaze

```
app.img_akaze = cv2.drawKeypoints(img, kp_akaze, None, color=(255, 0, 0))
```

5.2.2.29 img_orb

```
app.img_orb = cv2.drawKeypoints(img, kp_orb, None, color=(0, 255, 0))
```

5.2.2.30 img_sift

```
app.img_sift = cv2.drawKeypoints(img, kp_sift, None, color=(0, 0, 255))
```

5.2.2.31 IMG_SIZE

```
int app.IMG_SIZE = 224
```

5.2.2.32 img_surf

```
app.img_surf = cv2.drawKeypoints(img, kp_surf, None, color=(255, 255, 0))
```

5.2.2.33 kp_akaze

```
app.kp_akaze
```

5.2.2.34 kp_orb

```
app.kp_orb
```

5.2.2.35 kp_sift

```
app.kp_sift
```

5.2.2.36 kp_surf

```
app.kp_surf
```

5.2.2.37 layout

```
app.layout
```

5.2.2.38 lstm_fake_score

```
int app.lstm_fake_score = lstm_prediction[0][1] * 100
```

5.2.2.39 lstm_input

```
app.lstm_input = np.expand_dims(ai_image, axis=0)
```

5.2.2.40 lstm_model

```
app.lstm_model
```

5.2.2.41 lstm_prediction

```
app.lstm_prediction = lstm_model.predict(lstm_input, verbose=0)
```

5.2.2.42 lstm_real_score

```
int app.lstm_real_score = lstm_prediction[0][0] * 100
```

5.2.2.43 marked_img

```
app.marked_img = img.copy()
```

5.2.2.44 orb

```
app.orb = cv2.ORB_create()
```

5.2.2.45 page_icon

```
app.page_icon
```

5.2.2.46 page_title

```
app.page_title
```

5.2.2.47 sift

```
app.sift = cv2.SIFT_create()
```

5.2.2.48 surf

```
app.surf = cv2.xfeatures2d.SURF_create(400)
```

5.2.2.49 suspicious_count

```
int app.suspicious_count = 0
```

5.2.2.50 tab1

```
app.tab1
```

5.2.2.51 tab2

```
app.tab2
```

5.2.2.52 tab3

```
app.tab3
```

5.2.2.53 tab4

app.tab4

5.2.2.54 unsafe_allow_html

app.unsafe_allow_html

5.2.2.55 uploaded_file

app.uploaded_file

Initial value:

```
00001 = st.file_uploader(  
00002     "Analiz edilecek görüntüyü yükleyin",  
00003     type=["jpg", "jpeg", "png", "tif", "tiff"]  
00004 )
```

5.2.2.56 w

app.w

5.2.2.57 width

app.width

5.2.2.58 x

app.x

5.2.2.59 y

app.y

5.3 evaluate_models Namespace Reference

Functions

- [prepare_image](#) (path)
- [predict_image](#) (path)
- [evaluate_folder](#) (folder, expected_label, limit=100)

Variables

- int `IMG_SIZE` = 224
- `cnn_model` = `load_model("models/cnn_model.h5")`
- `lstm_model` = `load_model("models/lstm_model.h5")`
- `limit`

5.3.1 Function Documentation

5.3.1.1 `evaluate_folder()`

```
evaluate_models.evaluate_folder (  
    folder,  
    expected_label,  
    limit = 100)
```

5.3.1.2 `predict_image()`

```
evaluate_models.predict_image (  
    path)
```

5.3.1.3 `prepare_image()`

```
evaluate_models.prepare_image (  
    path)
```

5.3.2 Variable Documentation

5.3.2.1 `cnn_model`

```
evaluate_models.cnn_model = load_model("models/cnn_model.h5")
```

5.3.2.2 `IMG_SIZE`

```
int evaluate_models.IMG_SIZE = 224
```

5.3.2.3 `limit`

```
evaluate_models.limit
```

5.3.2.4 `lstm_model`

```
evaluate_models.lstm_model = load_model("models/lstm_model.h5")
```

5.4 feature_detection Namespace Reference

5.5 image_loader Namespace Reference

5.6 manual_model_check Namespace Reference

Functions

- [prepare_image](#) (path)
- [test_folder](#) (folder, true_label, count=30, [fake_index](#)=0)

Variables

- int [IMG_SIZE](#) = 160
- [cnn_model](#) = load_model("models/cnn_model.h5")
- [lstm_model](#) = load_model("models/lstm_model.h5")
- [fake_index](#)

5.6.1 Function Documentation

5.6.1.1 prepare_image()

```
manual_model_check.prepare_image (  
    path)
```

5.6.1.2 test_folder()

```
manual_model_check.test_folder (  
    folder,  
    true_label,  
    count = 30,  
    fake_index = 0)
```

5.6.2 Variable Documentation

5.6.2.1 cnn_model

```
manual_model_check.cnn_model = load_model("models/cnn_model.h5")
```

5.6.2.2 fake_index

```
manual_model_check.fake_index
```

5.6.2.3 IMG_SIZE

```
int manual_model_check.IMG_SIZE = 160
```

5.6.2.4 lstm_model

```
manual_model_check.lstm_model = load_model("models/lstm_model.h5")
```

5.7 metrics Namespace Reference

Functions

- [calculate_fake_ratio](#) (results)
- [interpret_result](#) (ratio)

5.7.1 Function Documentation

5.7.1.1 calculate_fake_ratio()

```
metrics.calculate_fake_ratio (  
    results)
```

```
results: ['real', 'fake', 'fake', ...]
```

5.7.1.2 interpret_result()

```
metrics.interpret_result (  
    ratio)
```

```
ratio: 0.0 - 1.0
```

5.8 model_test Namespace Reference

Functions

- [test_app_file_exists](#) ()

5.8.1 Function Documentation

5.8.1.1 test_app_file_exists()

```
model_test.test_app_file_exists ()
```

5.9 test_utils Namespace Reference

Functions

- [test_calculate_fake_ratio](#) ()
- [test_interpret_result](#) ()

5.9.1 Function Documentation

5.9.1.1 test_calculate_fake_ratio()

`test_utils.test_calculate_fake_ratio` ()

5.9.1.2 test_interpret_result()

`test_utils.test_interpret_result` ()

5.10 train_Istm Namespace Reference

Variables

- int [IMG_SIZE](#) = 224
- int [BATCH_SIZE](#) = 32
- int [EPOCHS](#) = 8
- str [DATASET_DIR](#) = "dataset"
- str [MODEL_DIR](#) = "models"
- [train_datagen](#)
- [train_generator](#)
- [val_generator](#)
- [classes](#) = [train_generator](#).classes
- [class_weights_array](#)
- [class_weights](#) = dict(enumerate([class_weights_array](#)))
- [fake_index](#) = [train_generator](#).class_indices["fake"]
- [base_model](#)
- [trainable](#)
- [x](#) = [base_model](#).output
- [output](#) = Dense(2, activation="softmax")(x)
- [model](#) = Model(inputs=[base_model](#).input, outputs=[output](#))
- [optimizer](#)
- [loss](#)
- [metrics](#)
- [validation_data](#)
- [epochs](#)
- [class_weight](#)

5.10.1 Variable Documentation

5.10.1.1 base_model

train_lstm.base_model

Initial value:

```
00001 = MobileNetV2(  
00002     weights="imagenet",  
00003     include_top=False,  
00004     input_shape=(IMG_SIZE, IMG_SIZE, 3)  
00005 )
```

5.10.1.2 BATCH_SIZE

```
int train_lstm.BATCH_SIZE = 32
```

5.10.1.3 class_weight

train_lstm.class_weight

5.10.1.4 class_weights

```
train_lstm.class_weights = dict(enumerate(class_weights_array))
```

5.10.1.5 class_weights_array

train_lstm.class_weights_array

Initial value:

```
00001 = compute_class_weight(  
00002     class_weight="balanced",  
00003     classes=np.unique(classes),  
00004     y=classes  
00005 )
```

5.10.1.6 classes

```
train_lstm.classes = train_generator.classes
```

5.10.1.7 DATASET_DIR

```
str train_lstm.DATASET_DIR = "dataset"
```

5.10.1.8 EPOCHS

```
int train_lstm.EPOCHS = 8
```

5.10.1.9 epochs

```
train_lstm.epochs
```

5.10.1.10 fake_index

```
train_lstm.fake_index = train_generator.class_indices["fake"]
```

5.10.1.11 IMG_SIZE

```
int train_lstm.IMG_SIZE = 224
```

5.10.1.12 loss

```
train_lstm.loss
```

5.10.1.13 metrics

```
train_lstm.metrics
```

5.10.1.14 model

```
train_lstm.model = Model(inputs=base_model.input, outputs=output)
```

5.10.1.15 MODEL_DIR

```
str train_lstm.MODEL_DIR = "models"
```

5.10.1.16 optimizer

```
train_lstm.optimizer
```

5.10.1.17 output

```
train_lstm.output = Dense(2, activation="softmax")(x)
```

5.10.1.18 train_datagen

train_lstm.train_datagen

Initial value:

```
00001 = ImageDataGenerator(  
00002     preprocessing_function=preprocess_input,  
00003     validation_split=0.2,  
00004     rotation_range=10,  
00005     zoom_range=0.1,  
00006     horizontal_flip=True,  
00007     brightness_range=[0.8, 1.2],  
00008     fill_mode="nearest"  
00009 )
```

5.10.1.19 train_generator

train_lstm.train_generator

Initial value:

```
00001 = train_datagen.flow_from_directory(  
00002     DATASET_DIR,  
00003     target_size=(IMG_SIZE, IMG_SIZE),  
00004     batch_size=BATCH_SIZE,  
00005     class_mode="categorical",  
00006     subset="training",  
00007     shuffle=True  
00008 )
```

5.10.1.20 trainable

train_lstm.trainable

5.10.1.21 val_generator

train_lstm.val_generator

Initial value:

```
00001 = train_datagen.flow_from_directory(  
00002     DATASET_DIR,  
00003     target_size=(IMG_SIZE, IMG_SIZE),  
00004     batch_size=BATCH_SIZE,  
00005     class_mode="categorical",  
00006     subset="validation",  
00007     shuffle=False  
00008 )
```

5.10.1.22 validation_data

train_lstm.validation_data

5.10.1.23 x

train_lstm.x = base_model.output

5.11 train_models Namespace Reference

Variables

- int [IMG_SIZE](#) = 224
- int [BATCH_SIZE](#) = 32
- int [EPOCHS_HEAD](#) = 15
- int [EPOCHS_FINE](#) = 15
- str [DATASET_DIR](#) = "dataset"
- str [MODEL_DIR](#) = "models"
- [exist_ok](#)
- [train_datagen](#)
- [train_generator](#)
- [val_generator](#)
- [classes](#) = train_generator.classes
- [class_weights_array](#)
- [class_weights](#) = dict(enumerate([class_weights_array](#)))
- [fake_index](#) = train_generator.class_indices["fake"]
- [base_model](#)
- [trainable](#)
- [x](#) = base_model.output
- [output](#) = Dense(2, activation="softmax")(x)
- [model](#) = Model(inputs=base_model.input, outputs=[output](#))
- [optimizer](#)
- [loss](#)
- [metrics](#)
- [validation_data](#)
- [epochs](#)
- [class_weight](#)

5.11.1 Variable Documentation

5.11.1.1 base_model

`train_models.base_model`

Initial value:

```
00001 = MobileNetV2(
00002     weights="imagenet",
00003     include_top=False,
00004     input_shape=(IMG_SIZE, IMG_SIZE, 3)
00005 )
```

5.11.1.2 BATCH_SIZE

`int train_models.BATCH_SIZE = 32`

5.11.1.3 class_weight

`train_models.class_weight`

5.11.1.4 class_weights

```
train_models.class_weights = dict(enumerate(class_weights_array))
```

5.11.1.5 class_weights_array

```
train_models.class_weights_array
```

Initial value:

```
00001 = compute_class_weight(  
00002     class_weight="balanced",  
00003     classes=np.unique(classes),  
00004     y=classes  
00005 )
```

5.11.1.6 classes

```
train_models.classes = train_generator.classes
```

5.11.1.7 DATASET_DIR

```
str train_models.DATASET_DIR = "dataset"
```

5.11.1.8 epochs

```
train_models.epochs
```

5.11.1.9 EPOCHS_FINE

```
int train_models.EPOCHS_FINE = 15
```

5.11.1.10 EPOCHS_HEAD

```
int train_models.EPOCHS_HEAD = 15
```

5.11.1.11 exist_ok

```
train_models.exist_ok
```

5.11.1.12 fake_index

```
train_models.fake_index = train_generator.class_indices["fake"]
```

5.11.1.13 IMG_SIZE

```
int train_models.IMG_SIZE = 224
```

5.11.1.14 loss

```
train_models.loss
```

5.11.1.15 metrics

```
train_models.metrics
```

5.11.1.16 model

```
train_models.model = Model(inputs=base_model.input, outputs=output)
```

5.11.1.17 MODEL_DIR

```
train_models.MODEL_DIR = "models"
```

5.11.1.18 optimizer

```
train_models.optimizer
```

5.11.1.19 output

```
train_models.output = Dense(2, activation="softmax")(x)
```

5.11.1.20 train_datagen

```
train_models.train_datagen
```

Initial value:

```
00001 = ImageDataGenerator(  
00002     preprocessing_function=preprocess_input,  
00003     validation_split=0.2,  
00004     rotation_range=10,  
00005     zoom_range=0.1,  
00006     horizontal_flip=True,  
00007     brightness_range=[0.8, 1.2],  
00008     fill_mode="nearest"  
00009 )
```

5.11.1.21 train_generator

`train_models.train_generator`

Initial value:

```
00001 = train_datagen.flow_from_directory(  
00002     DATASET_DIR,  
00003     target_size=(IMG_SIZE, IMG_SIZE),  
00004     batch_size=BATCH_SIZE,  
00005     class_mode="categorical",  
00006     subset="training",  
00007     shuffle=True  
00008 )
```

5.11.1.22 trainable

`train_models.trainable`

5.11.1.23 val_generator

`train_models.val_generator`

Initial value:

```
00001 = train_datagen.flow_from_directory(  
00002     DATASET_DIR,  
00003     target_size=(IMG_SIZE, IMG_SIZE),  
00004     batch_size=BATCH_SIZE,  
00005     class_mode="categorical",  
00006     subset="validation",  
00007     shuffle=False  
00008 )
```

5.11.1.24 validation_data

`train_models.validation_data`

5.11.1.25 x

`train_models.x = base_model.output`

5.12 utils Namespace Reference

Chapter 6

File Documentation

6.1 app.py File Reference

Namespaces

- namespace `app`

Functions

- `app.load_ai_models ()`

Variables

- `app.page_title`
- `app.page_icon`
- `app.layout`
- `int app.IMG_SIZE = 224`
- `app.cnn_model`
- `app.lstm_model`
- `app.unsafe_allow_html`
- `app.uploaded_file`
- `app.image = Image.open(uploaded_file).convert("RGB")`
- `app.img = np.array(image)`
- `app.gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)`
- `app.c1`
- `app.c2`
- `app.c3`
- `app.caption`
- `app.width`
- `app.tab1`
- `app.tab2`
- `app.tab3`
- `app.tab4`
- `app.orb = cv2.ORB_create()`
- `app.kp_orb`
- `app.des_orb`

- `app.img_orb = cv2.drawKeypoints(img, kp_orb, None, color=(0, 255, 0))`
- `app.akaze = cv2.AKAZE_create()`
- `app.kp_akaze`
- `app.des_akaze`
- `app.img_akaze = cv2.drawKeypoints(img, kp_akaze, None, color=(255, 0, 0))`
- `app.sift = cv2.SIFT_create()`
- `app.kp_sift`
- `app.des_sift`
- `app.img_sift = cv2.drawKeypoints(img, kp_sift, None, color=(0, 0, 255))`
- `app.surf = cv2.xfeatures2d.SURF_create(400)`
- `app.kp_surf`
- `app.des_surf`
- `app.img_surf = cv2.drawKeypoints(img, kp_surf, None, color=(255, 255, 0))`
- `app.edges = cv2.Canny(gray, 100, 200)`
- `app.contours`
- `app._`
- `app.marked_img = img.copy()`
- `int app.suspicious_count = 0`
- `app.area = cv2.contourArea(contour)`
- `app.x`
- `app.y`
- `app.w`
- `app.h`
- `app.ai_image = cv2.resize(img, (IMG_SIZE, IMG_SIZE))`
- `app.cnn_input = np.expand_dims(ai_image, axis=0)`
- `app.lstm_input = np.expand_dims(ai_image, axis=0)`
- `app.cnn_prediction = cnn_model.predict(cnn_input, verbose=0)`
- `app.lstm_prediction = lstm_model.predict(lstm_input, verbose=0)`
- `int app.cnn_real_score = cnn_prediction[0][0] * 100`
- `int app.cnn_fake_score = cnn_prediction[0][1] * 100`
- `int app.lstm_real_score = lstm_prediction[0][0] * 100`
- `int app.lstm_fake_score = lstm_prediction[0][1] * 100`
- `tuple app.final_score = (cnn_fake_score * 0.7) + (lstm_fake_score * 0.3)`
- `app.col1`
- `app.col2`
- `app.col3`

6.2 evaluate_models.py File Reference

Namespaces

- namespace `evaluate_models`

Functions

- `evaluate_models.prepare_image` (path)
- `evaluate_models.predict_image` (path)
- `evaluate_models.evaluate_folder` (folder, expected_label, limit=100)

Variables

- int `evaluate_models.IMG_SIZE` = 224
- `evaluate_models.cnn_model` = `load_model("models/cnn_model.h5")`
- `evaluate_models.lstm_model` = `load_model("models/lstm_model.h5")`
- `evaluate_models.limit`

6.3 manual_model_check.py File Reference

Namespaces

- namespace `manual_model_check`

Functions

- `manual_model_check.prepare_image` (path)
- `manual_model_check.test_folder` (folder, true_label, count=30, `fake_index`=0)

Variables

- int `manual_model_check.IMG_SIZE` = 160
- `manual_model_check.cnn_model` = `load_model("models/cnn_model.h5")`
- `manual_model_check.lstm_model` = `load_model("models/lstm_model.h5")`
- `manual_model_check.fake_index`

6.4 model_test.py File Reference

Namespaces

- namespace `model_test`

Functions

- `model_test.test_app_file_exists` ()

6.5 README.md File Reference

6.6 src/ai_detection.py File Reference

Namespaces

- namespace `ai_detection`

6.7 src/feature_detection.py File Reference

Namespaces

- namespace [feature_detection](#)

6.8 src/image_loader.py File Reference

Namespaces

- namespace [image_loader](#)

6.9 src/utils.py File Reference

Namespaces

- namespace [utils](#)

6.10 test_utils.py File Reference

Namespaces

- namespace [test_utils](#)

Functions

- [test_utils.test_calculate_fake_ratio](#) ()
- [test_utils.test_interpret_result](#) ()

6.11 train_lstm.py File Reference

Namespaces

- namespace [train_lstm](#)

Variables

- int `train_lstm.IMG_SIZE` = 224
- int `train_lstm.BATCH_SIZE` = 32
- int `train_lstm.EPOCHS` = 8
- str `train_lstm.DATASET_DIR` = "dataset"
- str `train_lstm.MODEL_DIR` = "models"
- `train_lstm.train_datagen`
- `train_lstm.train_generator`
- `train_lstm.val_generator`
- `train_lstm.classes` = `train_generator.classes`
- `train_lstm.class_weights_array`
- `train_lstm.class_weights` = `dict(enumerate(class_weights_array))`
- `train_lstm.fake_index` = `train_generator.class_indices["fake"]`
- `train_lstm.base_model`
- `train_lstm.trainable`
- `train_lstm.x` = `base_model.output`
- `train_lstm.output` = `Dense(2, activation="softmax")(x)`
- `train_lstm.model` = `Model(inputs=base_model.input, outputs=output)`
- `train_lstm.optimizer`
- `train_lstm.loss`
- `train_lstm.metrics`
- `train_lstm.validation_data`
- `train_lstm.epochs`
- `train_lstm.class_weight`

6.12 train_models.py File Reference**Namespaces**

- namespace `train_models`

Variables

- int `train_models.IMG_SIZE` = 224
- int `train_models.BATCH_SIZE` = 32
- int `train_models.EPOCHS_HEAD` = 15
- int `train_models.EPOCHS_FINE` = 15
- str `train_models.DATASET_DIR` = "dataset"
- str `train_models.MODEL_DIR` = "models"
- `train_models.exist_ok`
- `train_models.train_datagen`
- `train_models.train_generator`
- `train_models.val_generator`
- `train_models.classes` = `train_generator.classes`
- `train_models.class_weights_array`
- `train_models.class_weights` = `dict(enumerate(class_weights_array))`
- `train_models.fake_index` = `train_generator.class_indices["fake"]`
- `train_models.base_model`
- `train_models.trainable`
- `train_models.x` = `base_model.output`
- `train_models.output` = `Dense(2, activation="softmax")(x)`

- [train_models.model](#) = Model(inputs=base_model.input, outputs=[output](#))
- [train_models.optimizer](#)
- [train_models.loss](#)
- [train_models.metrics](#)
- [train_models.validation_data](#)
- [train_models.epochs](#)
- [train_models.class_weight](#)

6.13 utils/metrics.py File Reference

Namespaces

- namespace [metrics](#)

Functions

- [metrics.calculate_fake_ratio](#) (results)
- [metrics.interpret_result](#) (ratio)

Index

- - app, [10](#)
- ai_detection, [9](#)
- ai_image
 - app, [10](#)
- akaze
 - app, [10](#)
- app, [9](#)
 - _, [10](#)
 - ai_image, [10](#)
 - akaze, [10](#)
 - area, [11](#)
 - c1, [11](#)
 - c2, [11](#)
 - c3, [11](#)
 - caption, [11](#)
 - cnn_fake_score, [11](#)
 - cnn_input, [11](#)
 - cnn_model, [11](#)
 - cnn_prediction, [11](#)
 - cnn_real_score, [11](#)
 - col1, [12](#)
 - col2, [12](#)
 - col3, [12](#)
 - contours, [12](#)
 - des_akaze, [12](#)
 - des_orb, [12](#)
 - des_sift, [12](#)
 - des_surf, [12](#)
 - edges, [12](#)
 - final_score, [12](#)
 - gray, [13](#)
 - h, [13](#)
 - image, [13](#)
 - img, [13](#)
 - img_akaze, [13](#)
 - img_orb, [13](#)
 - img_sift, [13](#)
 - IMG_SIZE, [13](#)
 - img_surf, [13](#)
 - kp_akaze, [13](#)
 - kp_orb, [14](#)
 - kp_sift, [14](#)
 - kp_surf, [14](#)
 - layout, [14](#)
 - load_ai_models, [10](#)
 - lstm_fake_score, [14](#)
 - lstm_input, [14](#)
 - lstm_model, [14](#)
 - lstm_prediction, [14](#)
 - lstm_real_score, [14](#)
 - marked_img, [14](#)
 - orb, [15](#)
 - page_icon, [15](#)
 - page_title, [15](#)
 - sift, [15](#)
 - surf, [15](#)
 - suspicious_count, [15](#)
 - tab1, [15](#)
 - tab2, [15](#)
 - tab3, [15](#)
 - tab4, [15](#)
 - unsafe_allow_html, [16](#)
 - uploaded_file, [16](#)
 - w, [16](#)
 - width, [16](#)
 - x, [16](#)
 - y, [16](#)
- app.py, [29](#)
- area
 - app, [11](#)
- base_model
 - train_lstm, [21](#)
 - train_models, [24](#)
- BATCH_SIZE
 - train_lstm, [21](#)
 - train_models, [24](#)
- c1
 - app, [11](#)
- c2
 - app, [11](#)
- c3
 - app, [11](#)
- calculate_fake_ratio
 - metrics, [19](#)
- caption
 - app, [11](#)
- class_weight
 - train_lstm, [21](#)
 - train_models, [24](#)
- class_weights
 - train_lstm, [21](#)
 - train_models, [24](#)
- class_weights_array
 - train_lstm, [21](#)
 - train_models, [25](#)
- classes

- train_lstm, 21
 - train_models, 25
- cnn_fake_score
 - app, 11
- cnn_input
 - app, 11
- cnn_model
 - app, 11
 - evaluate_models, 17
 - manual_model_check, 18
- cnn_prediction
 - app, 11
- cnn_real_score
 - app, 11
- col1
 - app, 12
- col2
 - app, 12
- col3
 - app, 12
- contours
 - app, 12
- DATASET_DIR
 - train_lstm, 21
 - train_models, 25
- des_akaze
 - app, 12
- des_orb
 - app, 12
- des_sift
 - app, 12
- des_surf
 - app, 12
- edges
 - app, 12
- EPOCHS
 - train_lstm, 21
- epochs
 - train_lstm, 21
 - train_models, 25
- EPOCHS_FINE
 - train_models, 25
- EPOCHS_HEAD
 - train_models, 25
- evaluate_folder
 - evaluate_models, 17
- evaluate_models, 16
 - cnn_model, 17
 - evaluate_folder, 17
 - IMG_SIZE, 17
 - limit, 17
 - lstm_model, 17
 - predict_image, 17
 - prepare_image, 17
- evaluate_models.py, 30
- exist_ok
 - train_models, 25
- fake_index
 - manual_model_check, 18
 - train_lstm, 22
 - train_models, 25
- feature_detection, 18
- final_score
 - app, 12
- gray
 - app, 13
- h
 - app, 13
- image
 - app, 13
- image_loader, 18
- img
 - app, 13
- img_akaze
 - app, 13
- img_orb
 - app, 13
- img_sift
 - app, 13
- IMG_SIZE
 - app, 13
 - evaluate_models, 17
 - manual_model_check, 18
 - train_lstm, 22
 - train_models, 25
- img_surf
 - app, 13
- interpret_result
 - metrics, 19
- kp_akaze
 - app, 13
- kp_orb
 - app, 14
- kp_sift
 - app, 14
- kp_surf
 - app, 14
- layout
 - app, 14
- limit
 - evaluate_models, 17
- load_ai_models
 - app, 10
- loss
 - train_lstm, 22
 - train_models, 26
- lstm_fake_score
 - app, 14
- lstm_input
 - app, 14
- lstm_model

- app, 14
- evaluate_models, 17
- manual_model_check, 19
- lstm_prediction
 - app, 14
- lstm_real_score
 - app, 14
- manual_model_check, 18
 - cnn_model, 18
 - fake_index, 18
 - IMG_SIZE, 18
 - lstm_model, 19
 - prepare_image, 18
 - test_folder, 18
- manual_model_check.py, 31
- marked_img
 - app, 14
- metrics, 19
 - calculate_fake_ratio, 19
 - interpret_result, 19
 - train_lstm, 22
 - train_models, 26
- model
 - train_lstm, 22
 - train_models, 26
- MODEL_DIR
 - train_lstm, 22
 - train_models, 26
- model_test, 19
 - test_app_file_exists, 19
- model_test.py, 31
- optimizer
 - train_lstm, 22
 - train_models, 26
- orb
 - app, 15
- output
 - train_lstm, 22
 - train_models, 26
- page_icon
 - app, 15
- page_title
 - app, 15
- predict_image
 - evaluate_models, 17
- prepare_image
 - evaluate_models, 17
 - manual_model_check, 18
- README.md, 31
- sift
 - app, 15
- src Directory Reference, 7
- src/ai_detection.py, 31
- src/feature_detection.py, 32
- src/image_loader.py, 32
- src/utils.py, 32
- surf
 - app, 15
- suspicious_count
 - app, 15
- tab1
 - app, 15
- tab2
 - app, 15
- tab3
 - app, 15
- tab4
 - app, 15
- test_app_file_exists
 - model_test, 19
- test_calculate_fake_ratio
 - test_utils, 20
- test_folder
 - manual_model_check, 18
- test_interpret_result
 - test_utils, 20
- test_utils, 20
 - test_calculate_fake_ratio, 20
 - test_interpret_result, 20
- test_utils.py, 32
- train_datagen
 - train_lstm, 22
 - train_models, 26
- train_generator
 - train_lstm, 23
 - train_models, 26
- train_lstm, 20
 - base_model, 21
 - BATCH_SIZE, 21
 - class_weight, 21
 - class_weights, 21
 - class_weights_array, 21
 - classes, 21
 - DATASET_DIR, 21
 - EPOCHS, 21
 - epochs, 21
 - fake_index, 22
 - IMG_SIZE, 22
 - loss, 22
 - metrics, 22
 - model, 22
 - MODEL_DIR, 22
 - optimizer, 22
 - output, 22
 - train_datagen, 22
 - train_generator, 23
 - trainable, 23
 - val_generator, 23
 - validation_data, 23
 - x, 23
- train_lstm.py, 32
- train_models, 24

- base_model, [24](#)
- BATCH_SIZE, [24](#)
- class_weight, [24](#)
- class_weights, [24](#)
- class_weights_array, [25](#)
- classes, [25](#)
- DATASET_DIR, [25](#)
- epochs, [25](#)
- EPOCHS_FINE, [25](#)
- EPOCHS_HEAD, [25](#)
- exist_ok, [25](#)
- fake_index, [25](#)
- IMG_SIZE, [25](#)
- loss, [26](#)
- metrics, [26](#)
- model, [26](#)
- MODEL_DIR, [26](#)
- optimizer, [26](#)
- output, [26](#)
- train_datagen, [26](#)
- train_generator, [26](#)
- trainable, [27](#)
- val_generator, [27](#)
- validation_data, [27](#)
- x, [27](#)
- train_models.py, [33](#)
- trainable
 - train_lstm, [23](#)
 - train_models, [27](#)
- unsafe_allow_html
 - app, [16](#)
- uploaded_file
 - app, [16](#)
- utils, [27](#)
- utils Directory Reference, [7](#)
- utils/metrics.py, [34](#)
- val_generator
 - train_lstm, [23](#)
 - train_models, [27](#)
- validation_data
 - train_lstm, [23](#)
 - train_models, [27](#)
- w
 - app, [16](#)
- width
 - app, [16](#)
- x
 - app, [16](#)
 - train_lstm, [23](#)
 - train_models, [27](#)
- y
 - app, [16](#)